

```
In [1]: import pandas as pd;
import numpy as np;
import seaborn as sns;
import matplotlib.pyplot as plt;
data=pd.read_csv(r"E:\pradnya's programming\Domains\DataAnalytics\DA_clgAss\Loan
data
```

```
Out[1]:
```

	age	employ	address	income	debtinc	creddebt	othdebt	ed	default
0	41.0	17	12	176.0	9.3	11.359392	5.008608	3.0	1
1	27.0	10	6	31.0	17.3	1.362202	4.000798	1.0	0
2	40.0	15	7	NaN	5.5	0.856075	2.168925	1.0	0
3	41.0	15	14	120.0	2.9	2.658720	0.821280	NaN	0
4	24.0	2	0	28.0	17.3	1.787436	3.056564	2.0	1
...	...	...	...	...	...	...	...	...	...
695	36.0	6	15	27.0	4.6	0.262062	0.979938	2.0	1
696	29.0	6	4	21.0	11.5	0.369495	2.045505	2.0	0
697	33.0	15	3	32.0	7.6	0.491264	1.940736	1.0	0
698	45.0	19	22	77.0	8.4	2.302608	4.165392	1.0	0
699	37.0	12	14	NaN	14.7	2.994684	3.473316	1.0	0

700 rows × 9 columns

```
In [2]: data.head()
```

```
Out[2]:
```

	age	employ	address	income	debtinc	creddebt	othdebt	ed	default
0	41.0	17	12	176.0	9.3	11.359392	5.008608	3.0	1
1	27.0	10	6	31.0	17.3	1.362202	4.000798	1.0	0
2	40.0	15	7	NaN	5.5	0.856075	2.168925	1.0	0
3	41.0	15	14	120.0	2.9	2.658720	0.821280	NaN	0
4	24.0	2	0	28.0	17.3	1.787436	3.056564	2.0	1

```
In [3]: data.shape
```

```
Out[3]: (700, 9)
```

```
In [4]: data.isnull().sum()
```

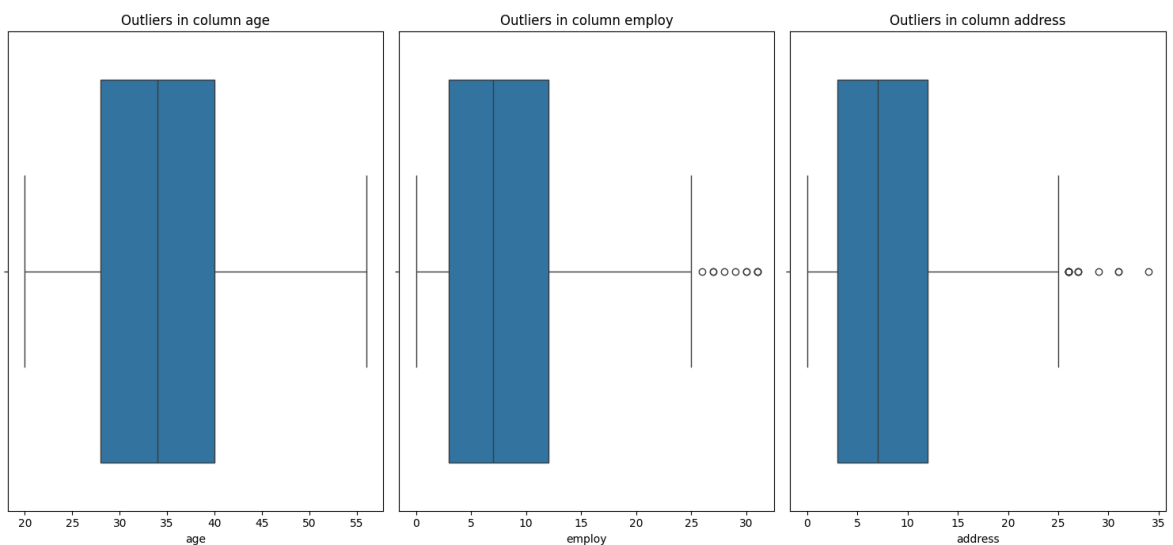
```
Out[4]: age      20
        employ    0
        address    0
        income    37
        debtinc    0
        creddebt    0
        othdebt    0
        ed        20
        default    0
        dtype: int64
```

```
In [5]: plt.figure(figsize=(15,7))
        plt.subplot(1, 3, 1)
        sns.boxplot(x=data["age"])
        plt.title("Outliers in column age")

        plt.subplot(1, 3, 2)
        sns.boxplot(x=data["employ"])
        plt.title("Outliers in column employ")

        plt.subplot(1, 3, 3)
        sns.boxplot(x=data["address"])
        plt.title("Outliers in column address")

        plt.tight_layout()
        plt.show()
```

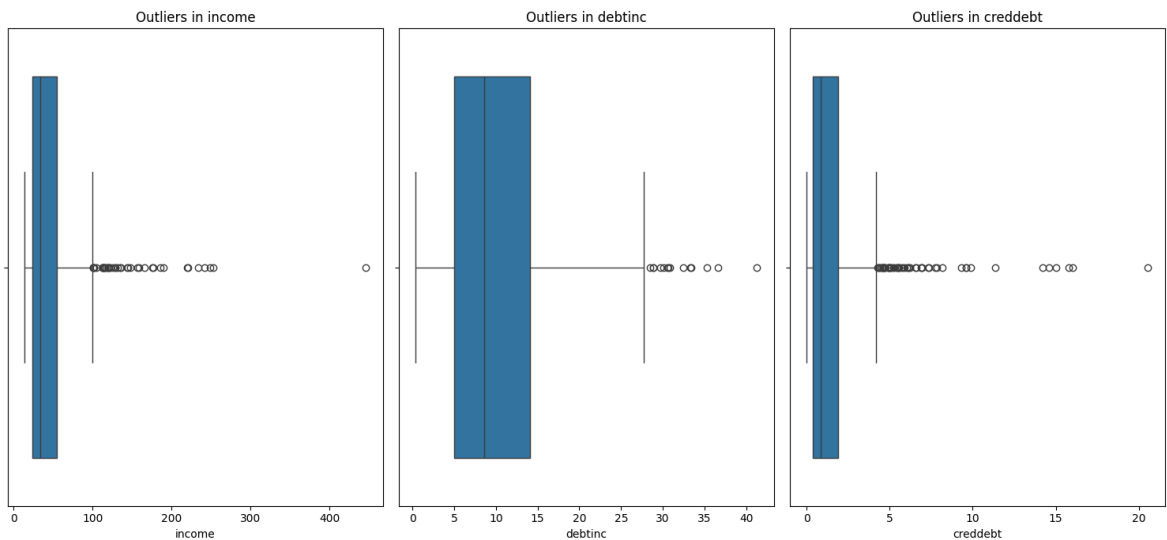


```
In [6]: plt.figure(figsize=(15,7))
        plt.subplot(1, 3, 1)
        sns.boxplot(x=data["income"])
        plt.title("Outliers in income")

        plt.subplot(1, 3, 2)
        sns.boxplot(x=data["debtinc"])
        plt.title("Outliers in debtinc")

        plt.subplot(1, 3, 3)
        sns.boxplot(x=data["creddebt"])
        plt.title("Outliers in creddebt")
```

```
plt.tight_layout()
plt.show()
```



```
In [8]: # missing values
print("New data frame length:", len(data))
data.isna().sum()

data = data.dropna(axis=0)
print("New data frame length:", len(data))
```

New data frame length: 625
New data frame length: 625

```
In [9]: print("\nMissing Values After Handling:")
print(data.isnull().sum())
```

Missing Values After Handling:

```
age      0
employ   0
address  0
income   0
debtinc  0
creddebt 0
othdebt  0
ed        0
default  0
dtype: int64
```

```
In [10]: def remove_outliers_iqr(df, column):
Q1 = df[column].quantile(0.25)
Q3 = df[column].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
return df[(df[column] >= lower) & (df[column] <= upper)]
```

```
In [11]: data = remove_outliers_iqr(data, 'age')
data = remove_outliers_iqr(data, 'employ')
data = remove_outliers_iqr(data, 'address')
data = remove_outliers_iqr(data, 'income')
data = remove_outliers_iqr(data, 'debtinc')
data = remove_outliers_iqr(data, 'creddebt')
data = remove_outliers_iqr(data, 'ed')
```

```
data = remove_outliers_iqr(data, 'default')
print(f"\nData Shape After Outlier Removal: {data.shape}")
```

Data Shape After Outlier Removal: (374, 9)

Feature Transformation (Scaling)

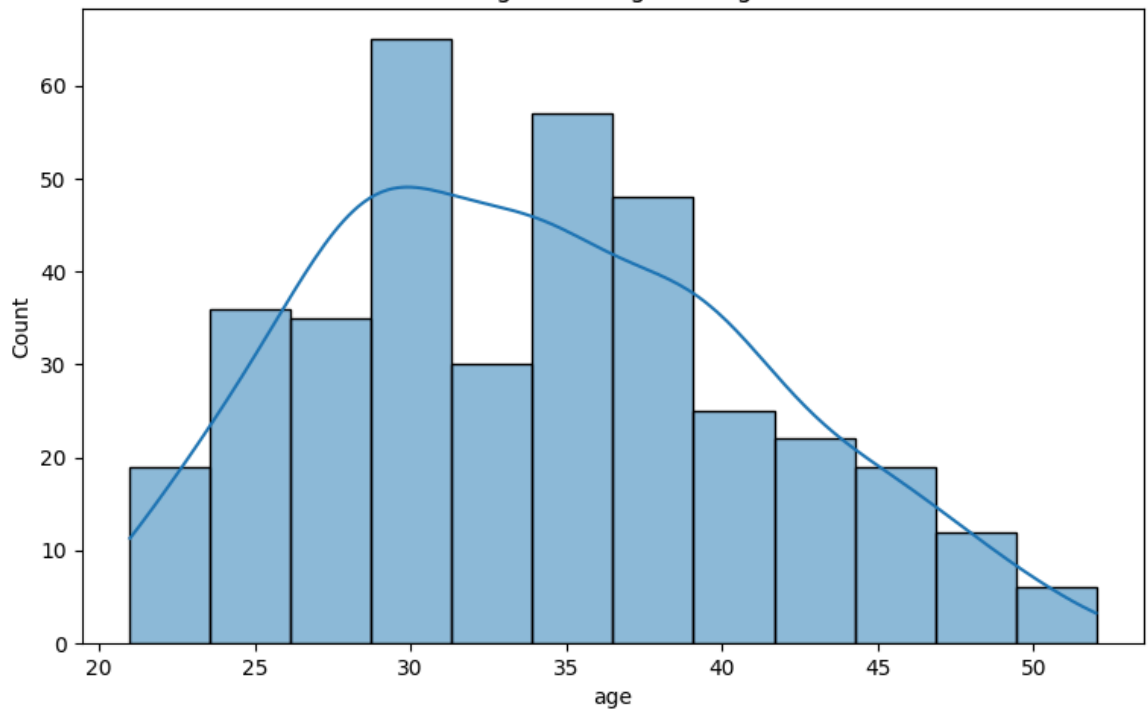
```
In [12]: def plot_distribution(data, columns, title_prefix="Original"):
plt.figure(figsize=(15, 5 * len(columns)))
for i, col in enumerate(columns, 1):
    plt.subplot(len(columns), 2, i * 2 - 1)
    sns.histplot(data[col], kde=True)
    plt.title(f"{title_prefix} Histogram - {col}")

plt.tight_layout()
plt.show()

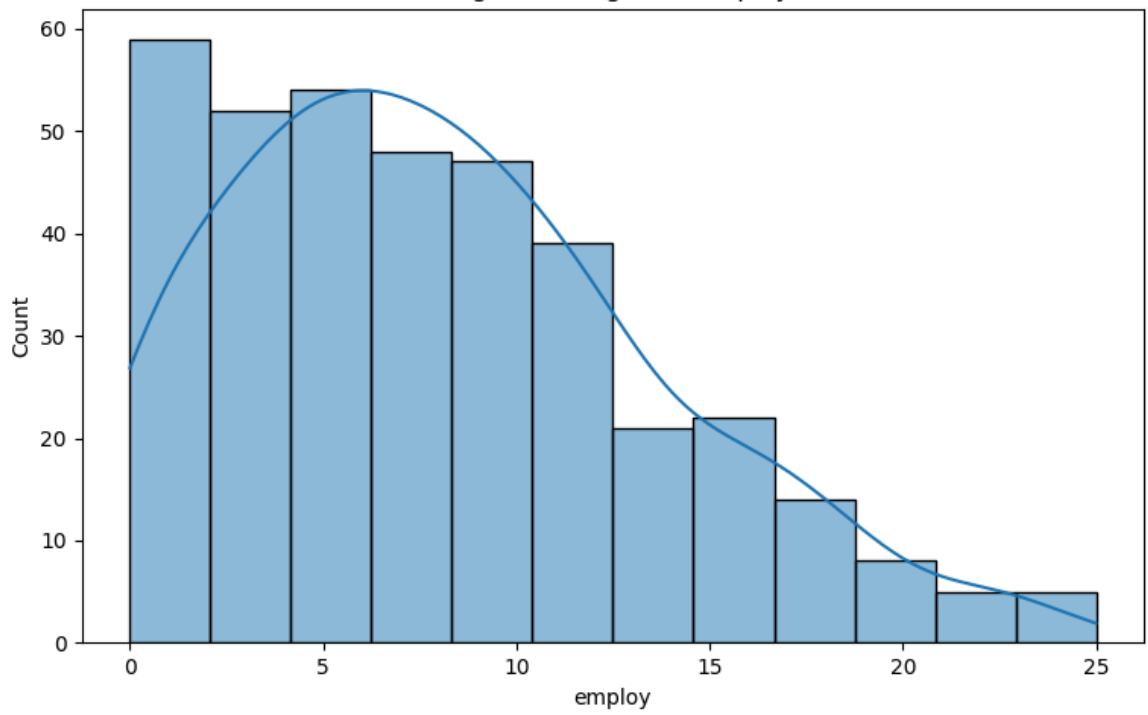
# Plot original distributions

numeric_columns = data.select_dtypes(include=np.number).columns.tolist()
plot_distribution(data, numeric_columns, title_prefix="Original")
```

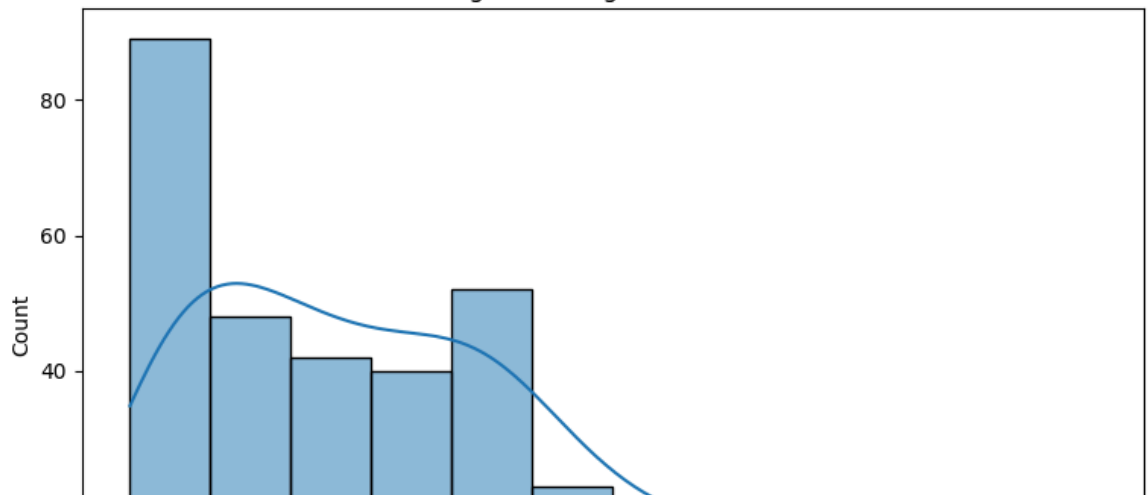
Original Histogram - age

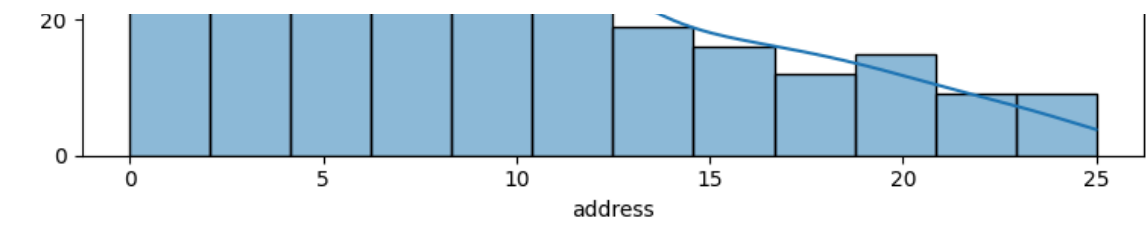


Original Histogram - employ

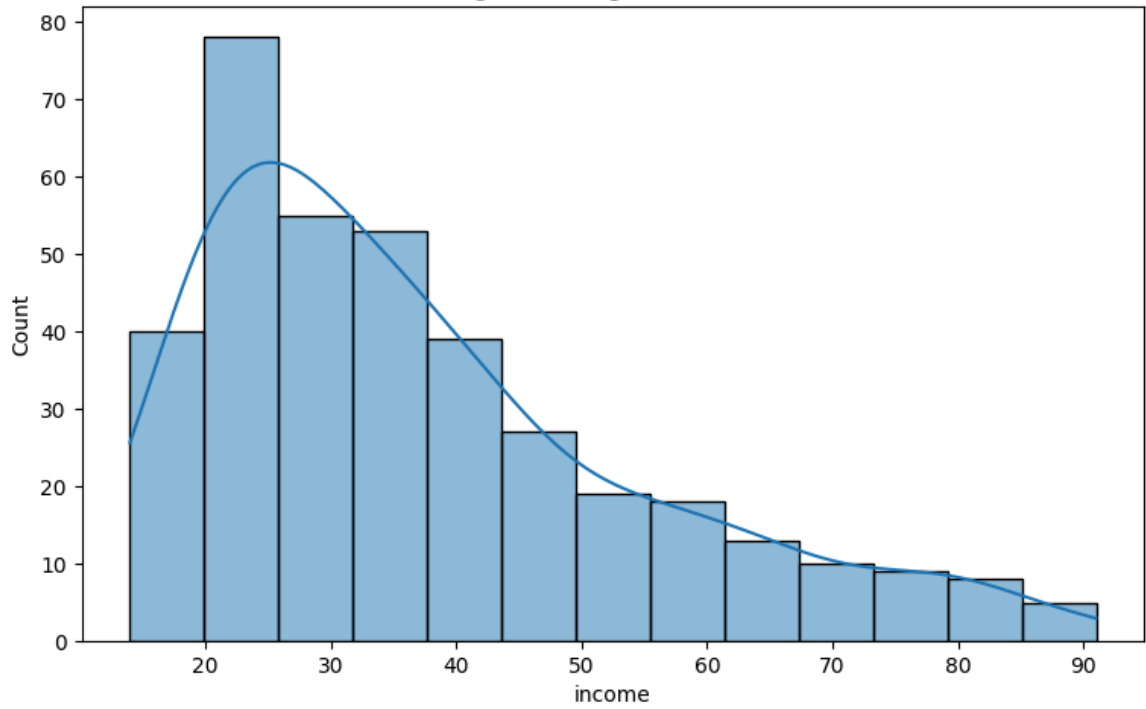


Original Histogram - address

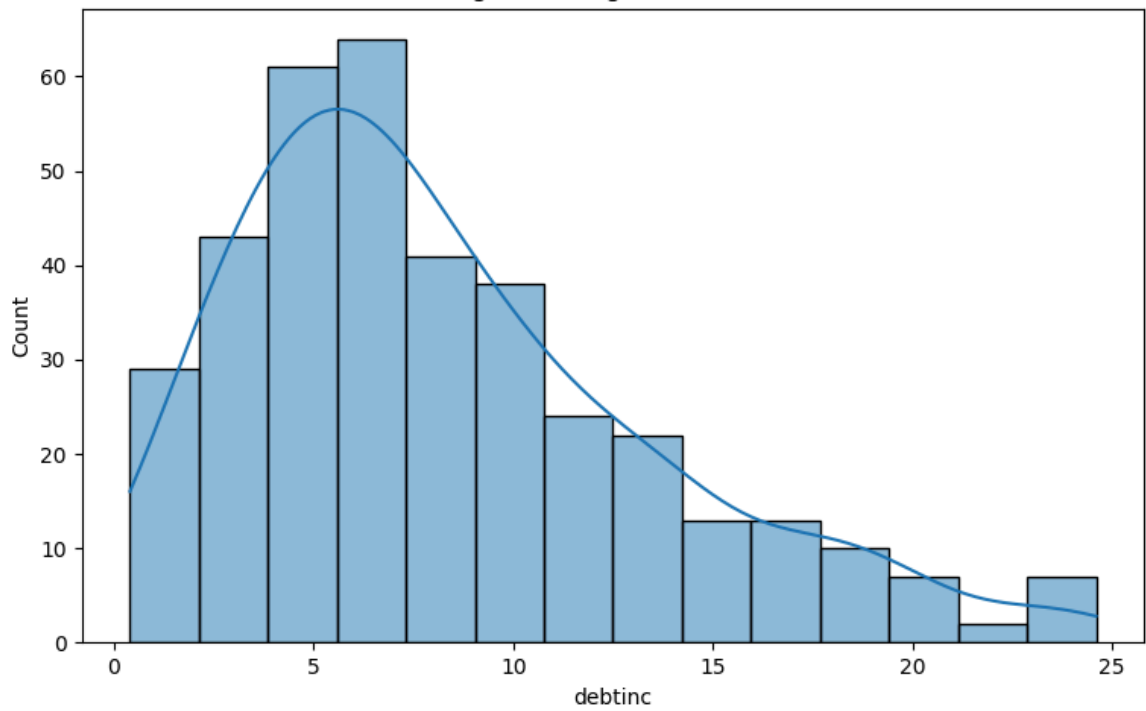




Original Histogram - income

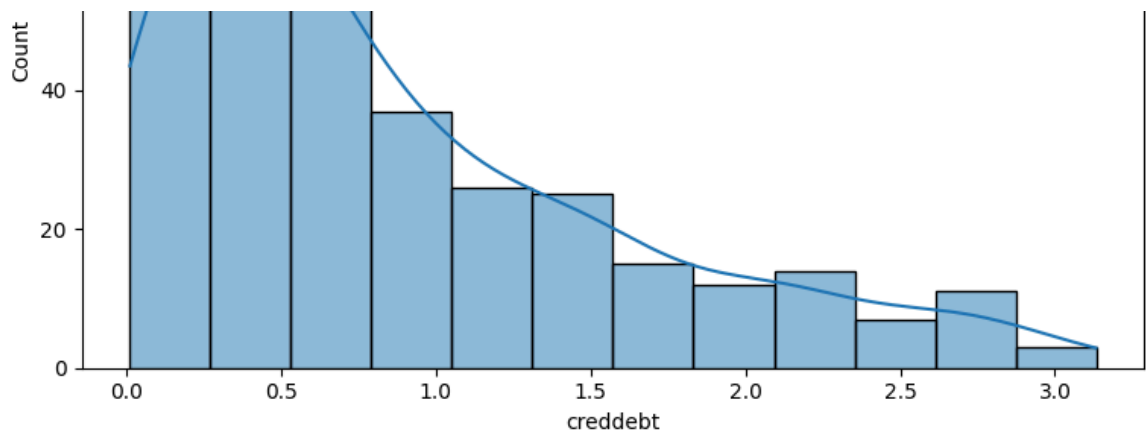


Original Histogram - debtinc

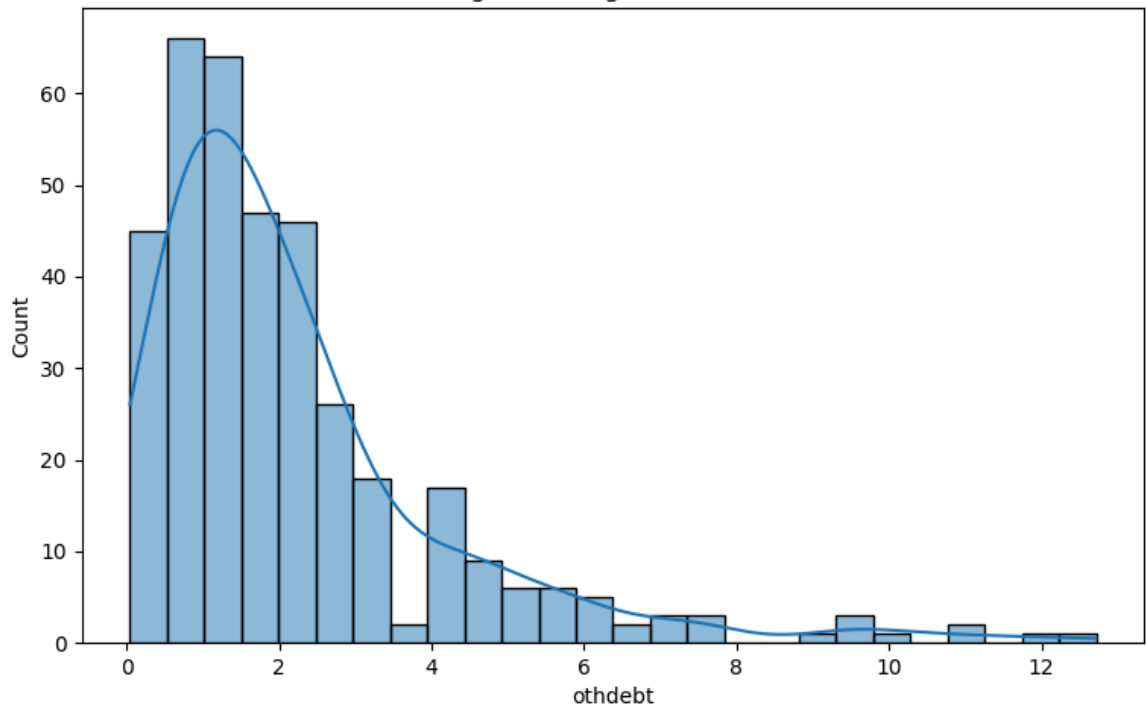


Original Histogram - creddebt

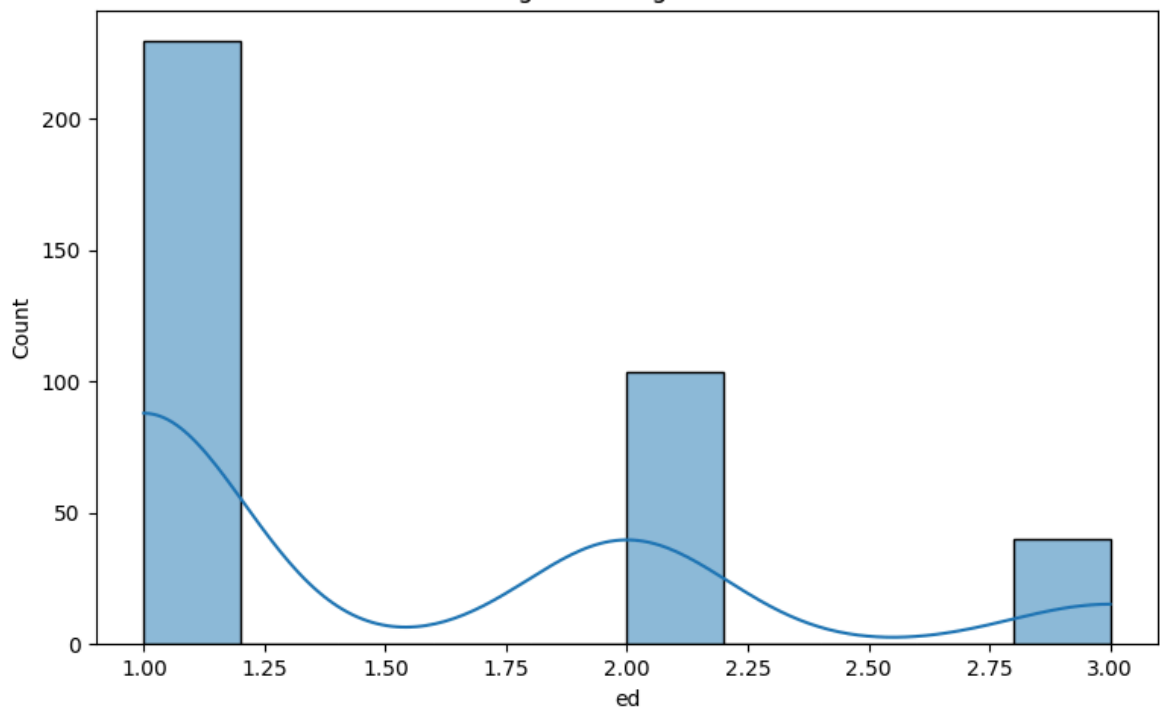




Original Histogram - othdebt

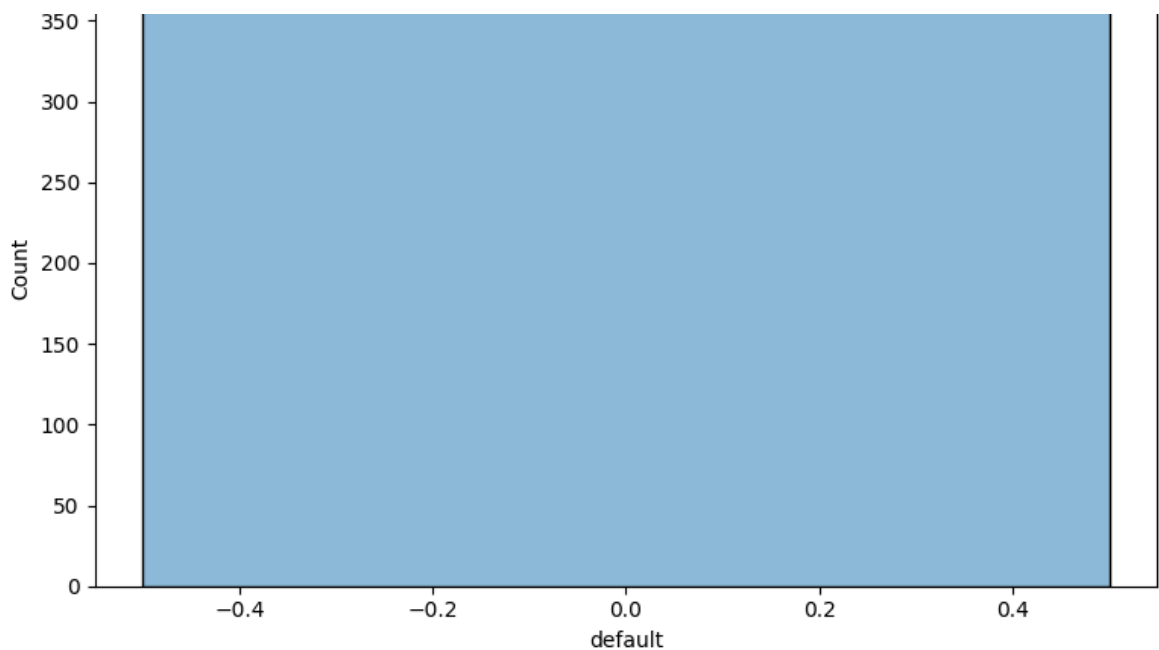


Original Histogram - ed



Original Histogram - default





```
In [13]: from sklearn.preprocessing import MinMaxScaler, StandardScaler

scaler_minmax = MinMaxScaler()
data_minmax = data.copy()
data_minmax[numeric_columns] = scaler_minmax.fit_transform(data[numeric_columns])
print("\nMin-Max Scaled Data Sample:")
print(data_minmax.head())

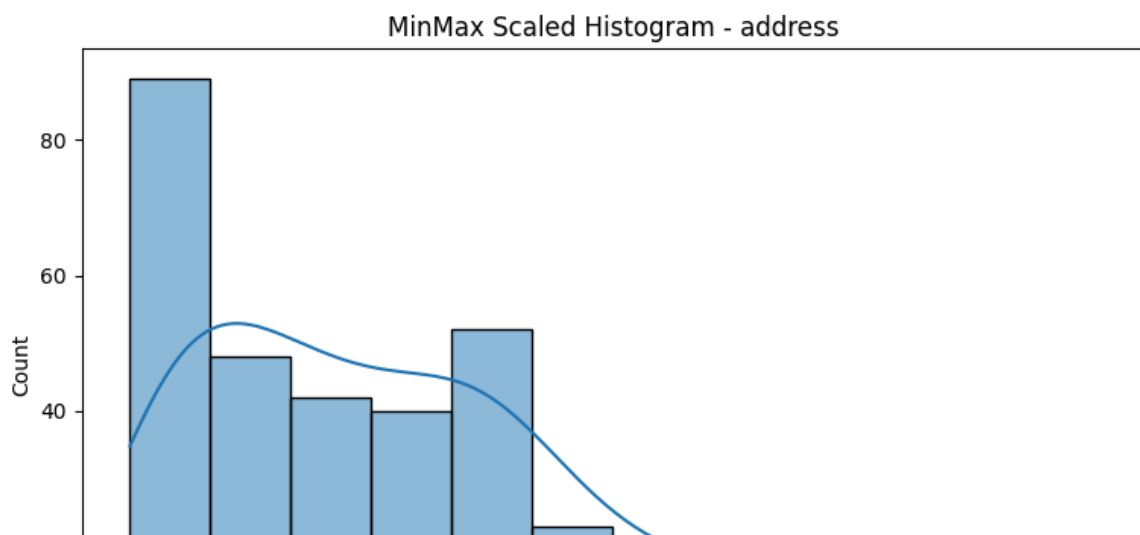
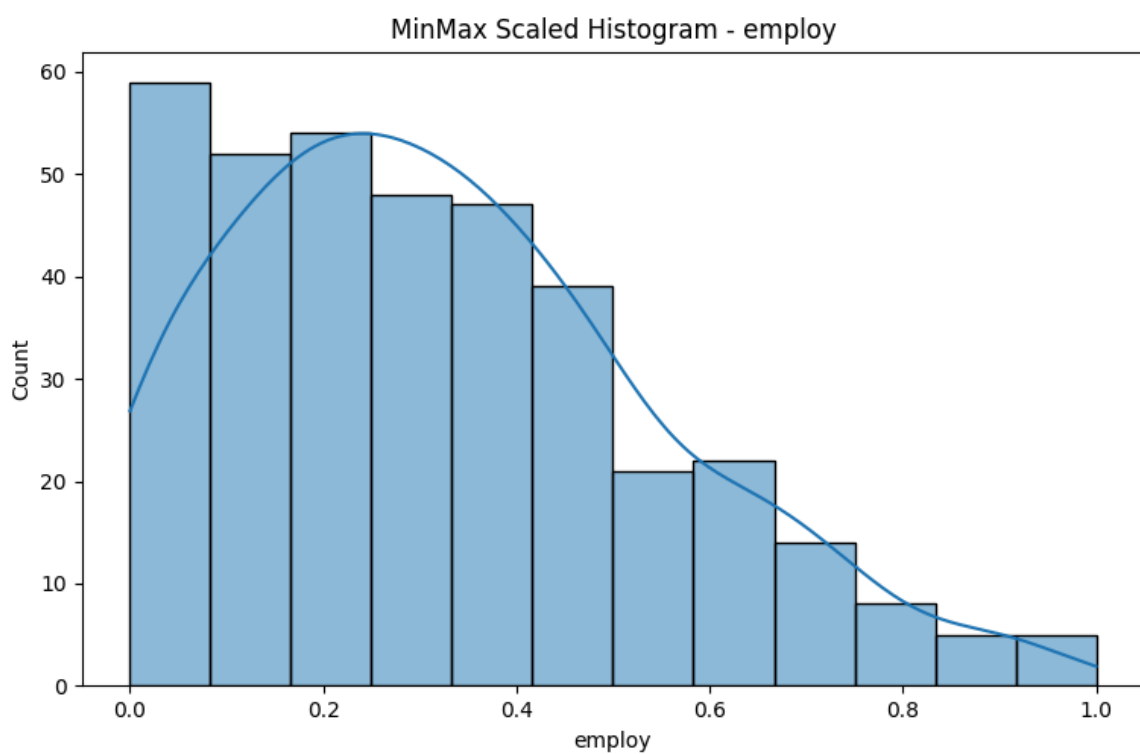
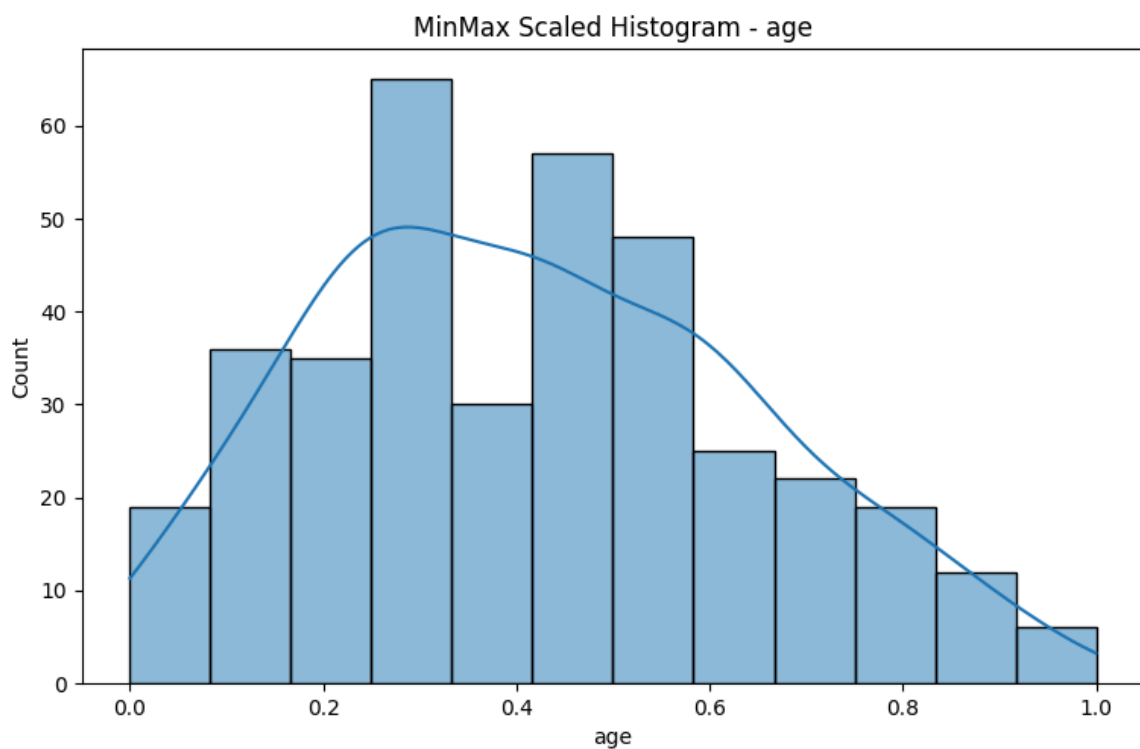
plot_distribution(data_minmax, numeric_columns, title_prefix="MinMax Scaled")
```

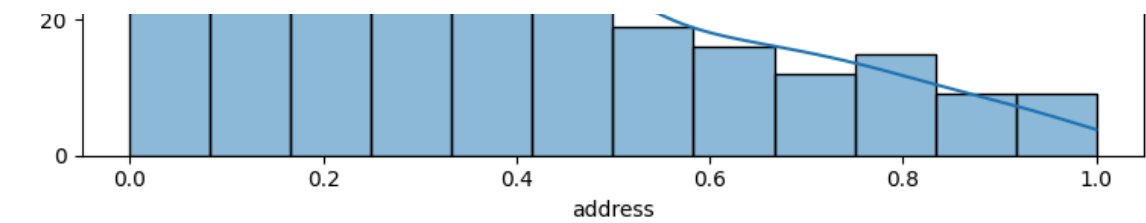
Min-Max Scaled Data Sample:

	age	employ	address	income	debtinc	creddebt	othdebt	ed \
1	0.193548	0.40	0.24	0.220779	0.698347	0.432719	0.312210	0.0
5	0.645161	0.20	0.20	0.142857	0.404959	0.122078	0.166691	0.5
9	0.483871	0.00	0.52	0.142857	0.797521	0.886262	0.165902	0.0
10	0.193548	0.00	0.04	0.025974	0.053719	0.054732	0.003466	0.0
11	0.129032	0.16	0.00	0.116883	0.198347	0.077110	0.070890	0.0

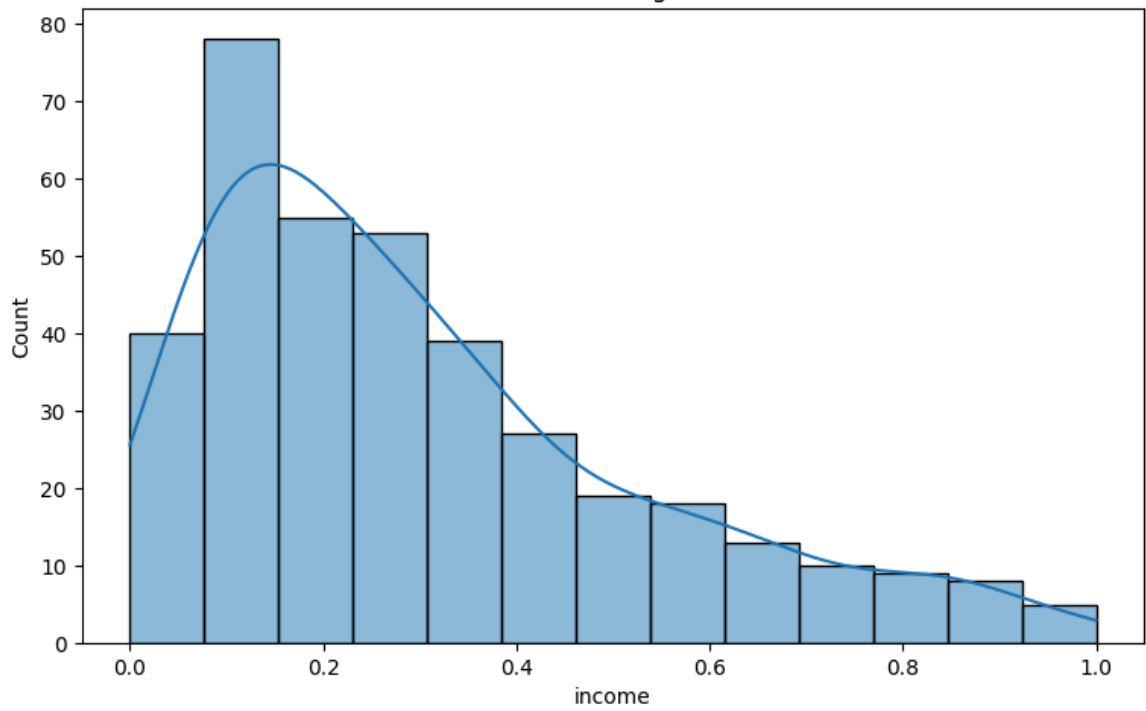
default

1	0.0
5	0.0
9	0.0
10	0.0
11	0.0

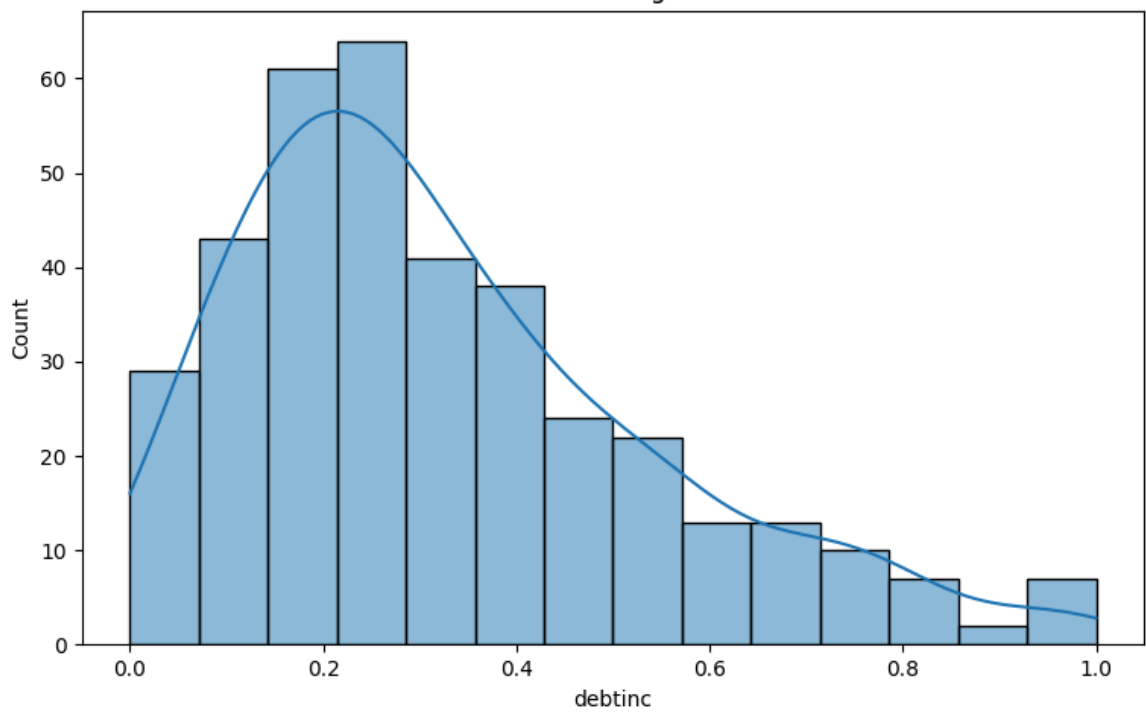




MinMax Scaled Histogram - income

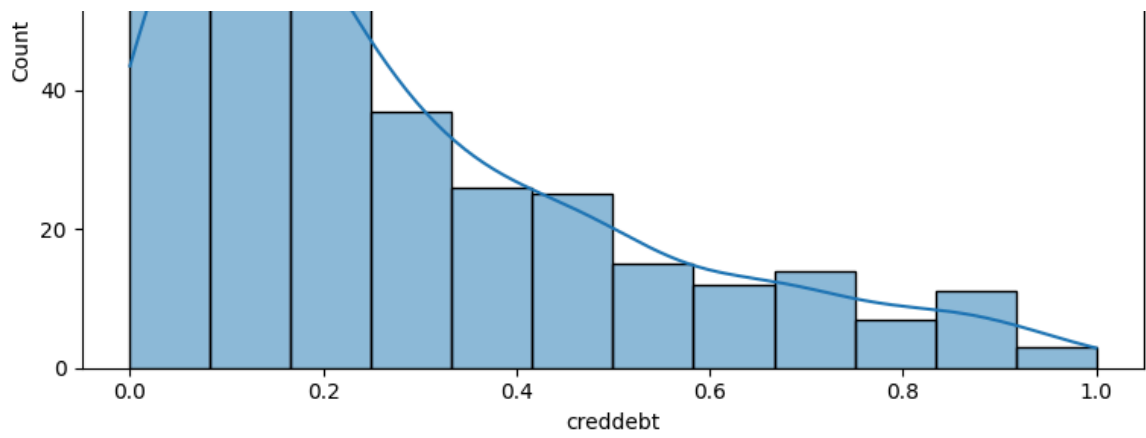


MinMax Scaled Histogram - debtinc

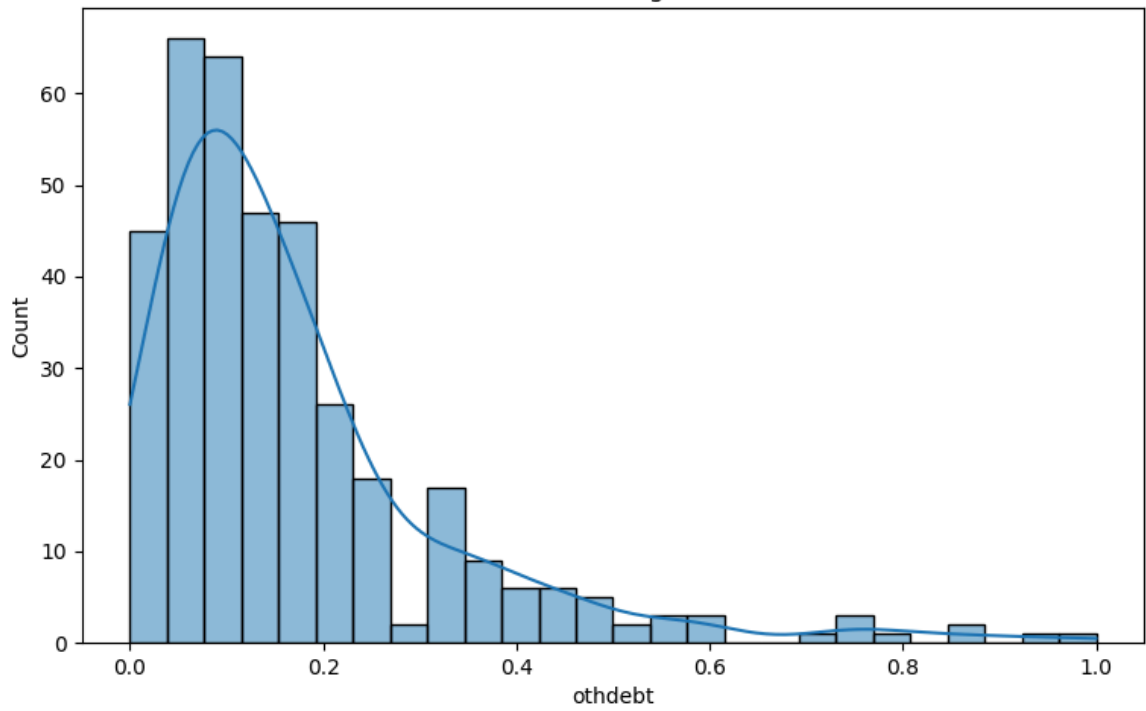


MinMax Scaled Histogram - creddebt

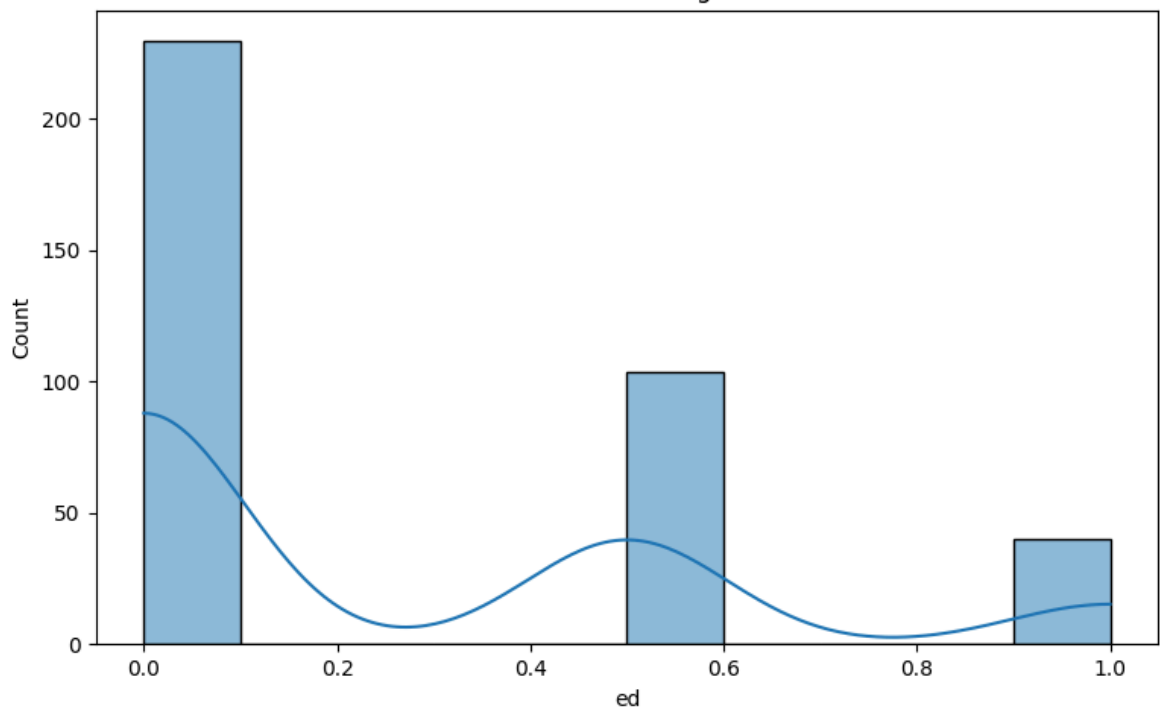




MinMax Scaled Histogram - othdebt

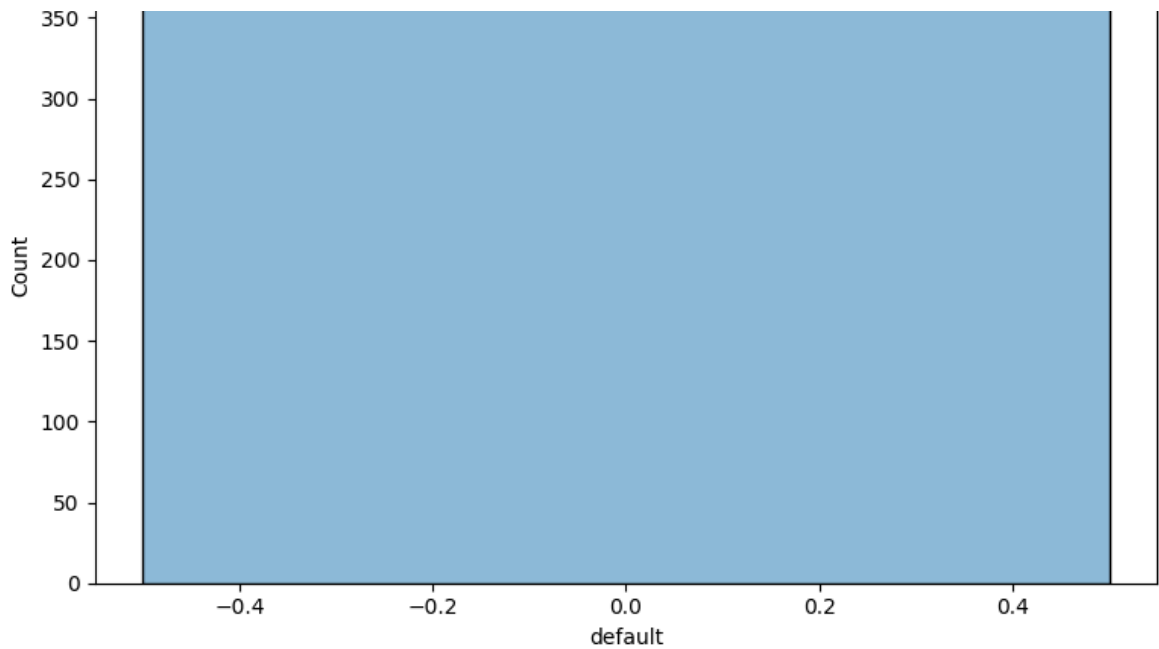


MinMax Scaled Histogram - ed



MinMax Scaled Histogram - default





```
In [14]: scaler_standard = StandardScaler()
data_standard = data.copy()
data_standard[numeric_columns] = scaler_standard.fit_transform(data[numeric_columns])

print("\nStandard Scaled Data Sample:")
print(data_standard.head())

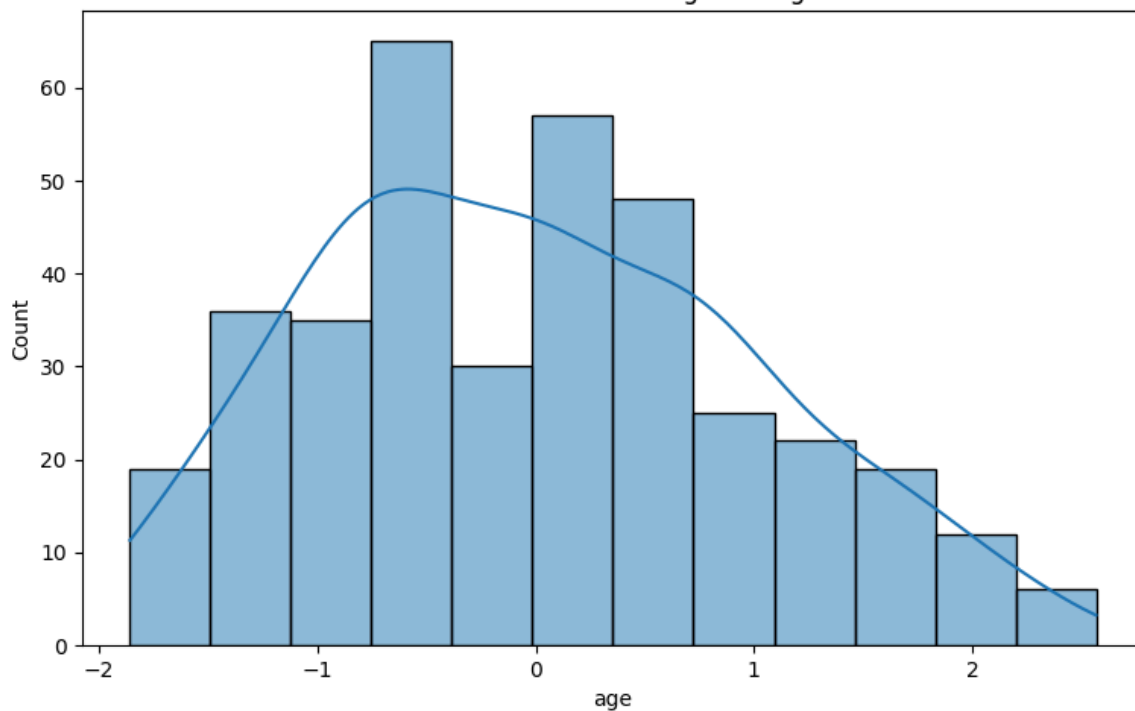
# Visualize after Standard Scaling
plot_distribution(data_standard, numeric_columns, title_prefix="Standard Scaled")
```

Standard Scaled Data Sample:

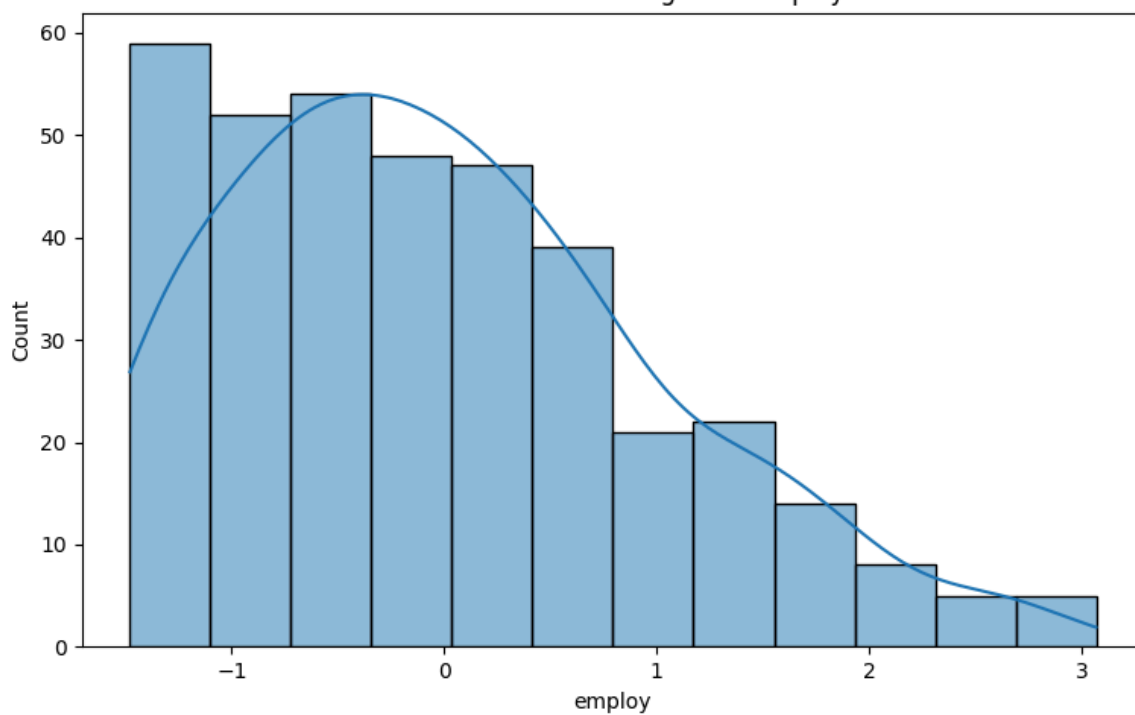
	age	employ	address	income	debtinc	creddebt	othdebt	\
1	-1.001558	0.339649	-0.314649	-0.364019	1.718720	0.698934	0.882385	
5	0.996216	-0.568997	-0.477413	-0.702965	0.360460	-0.620319	-0.026594	
9	0.282725	-1.477643	0.824702	-0.702965	2.177850	2.625077	-0.031524	
10	-1.001558	-1.477643	-1.128471	-1.211383	-1.265625	-0.906333	-1.046175	
11	-1.286954	-0.750726	-1.291236	-0.815947	-0.596061	-0.811293	-0.625014	

	ed	default
1	-0.722374	0.0
5	0.745930	0.0
9	-0.722374	0.0
10	-0.722374	0.0
11	-0.722374	0.0

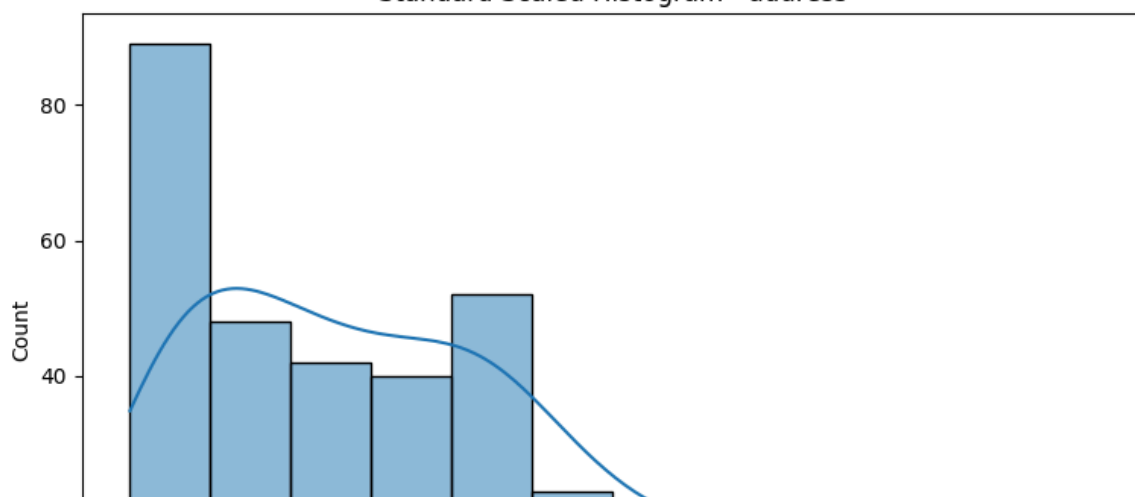
Standard Scaled Histogram - age

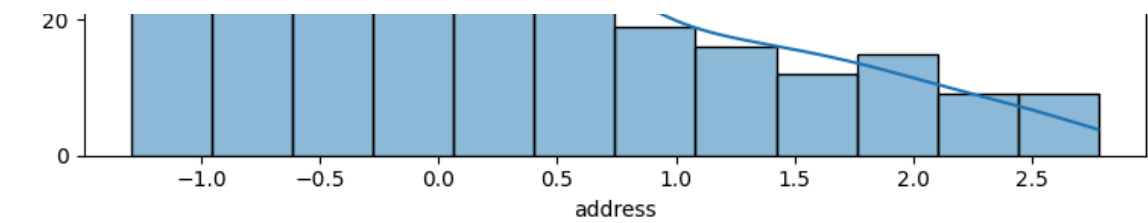


Standard Scaled Histogram - employ

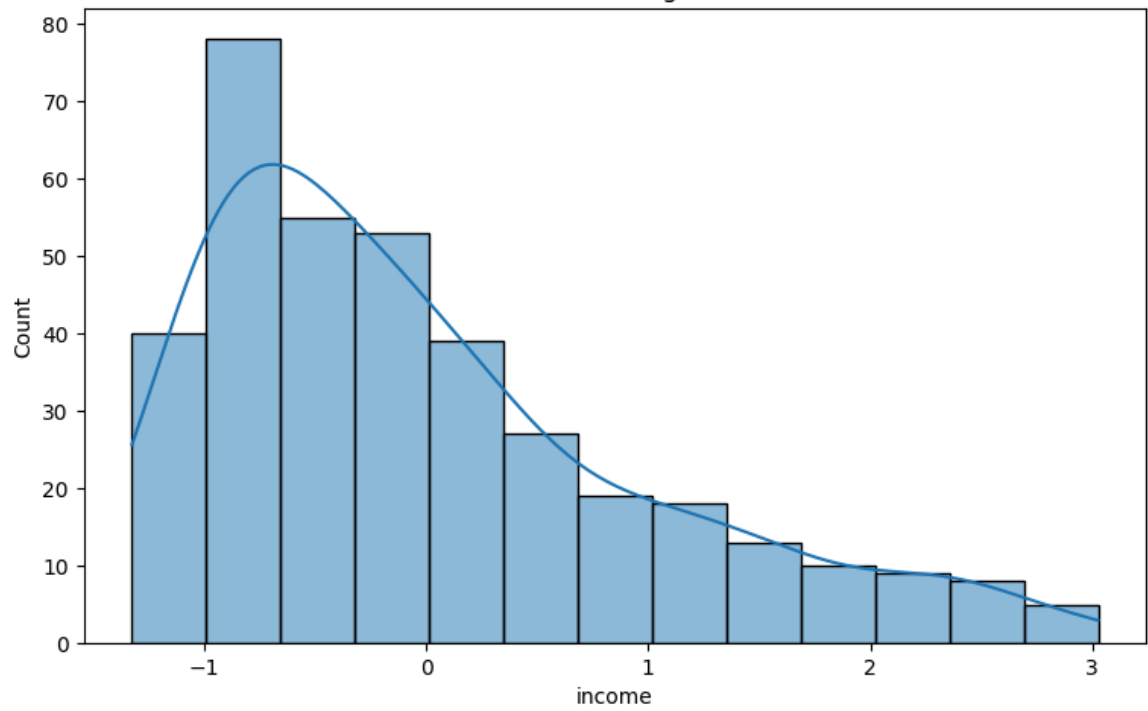


Standard Scaled Histogram - address

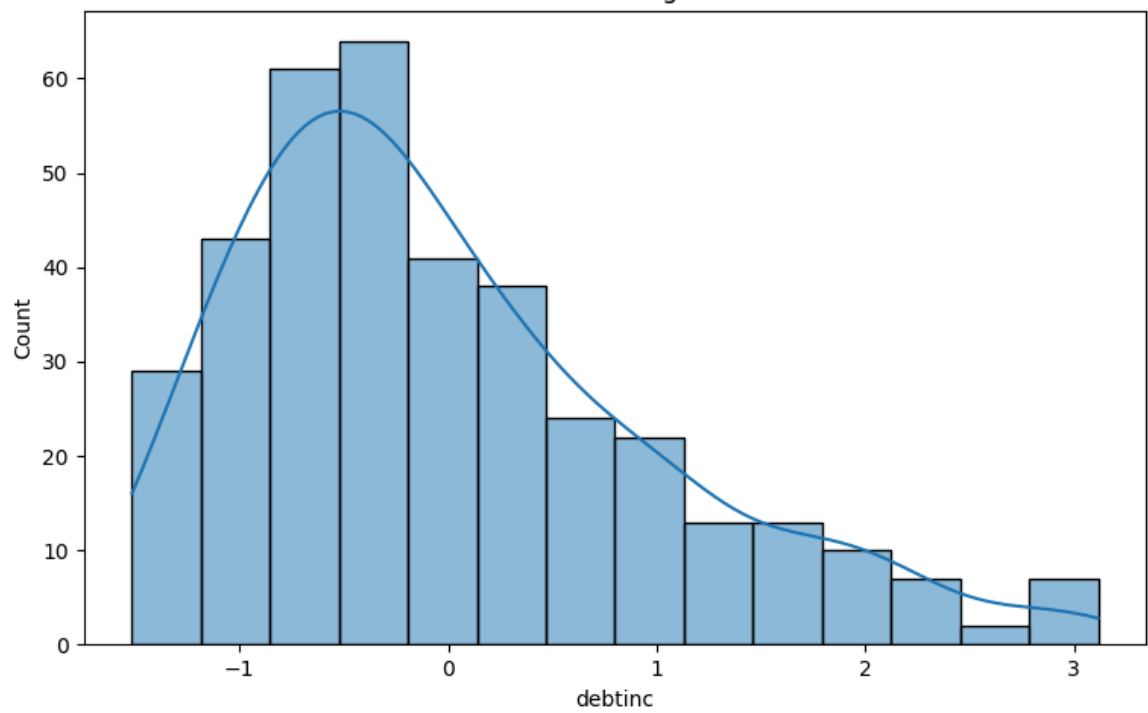




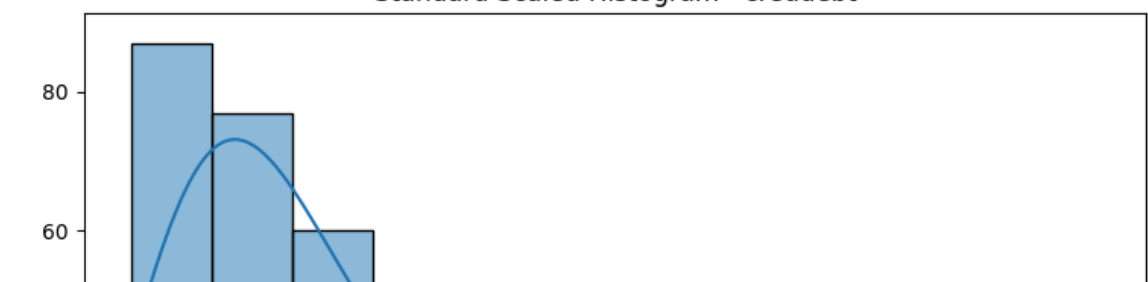
Standard Scaled Histogram - income

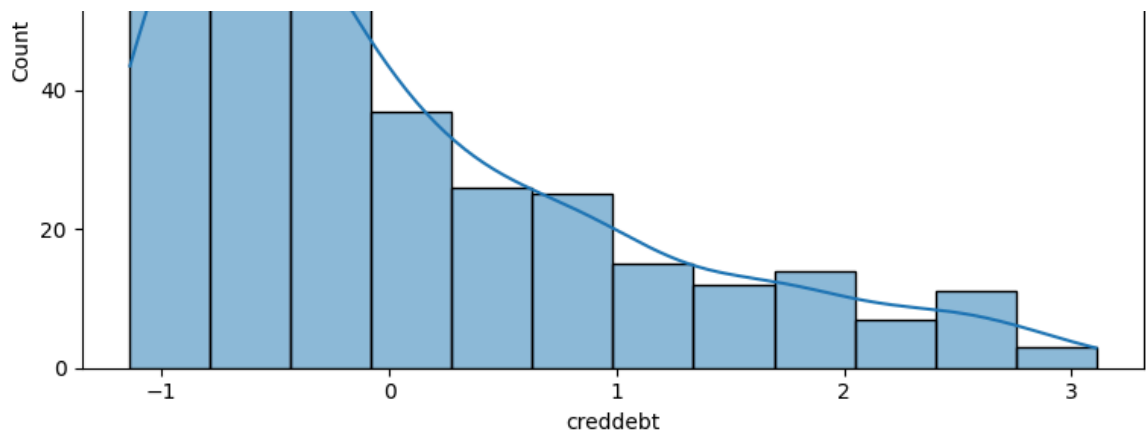


Standard Scaled Histogram - debtinc

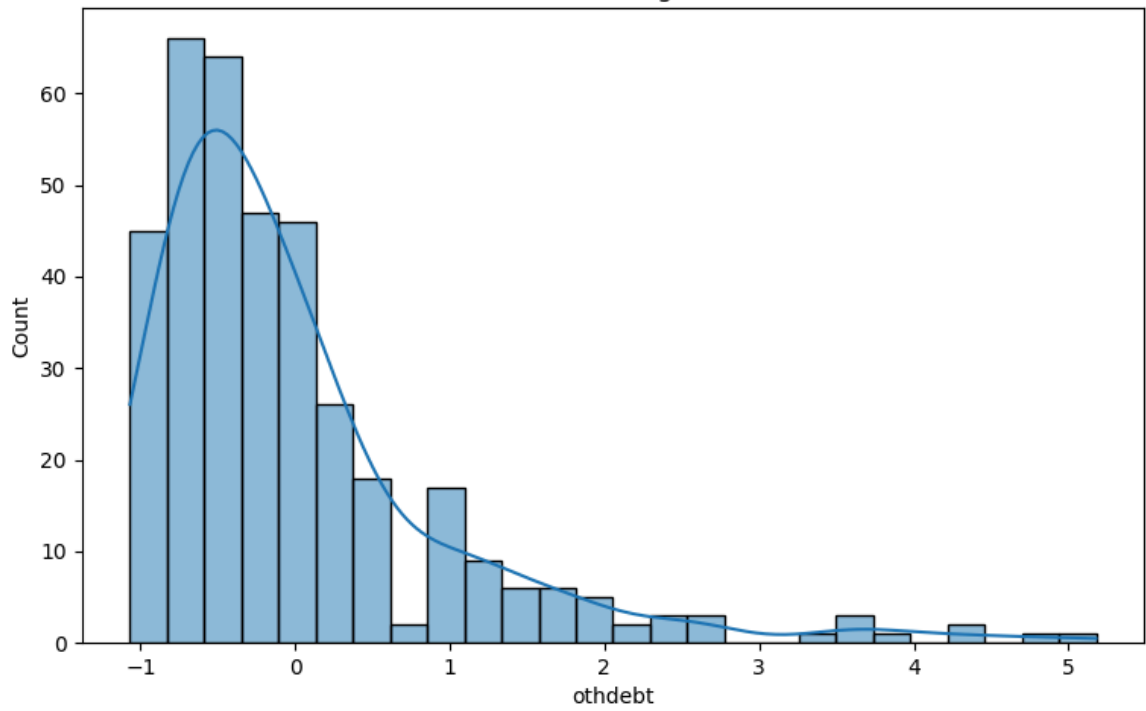


Standard Scaled Histogram - creddebt

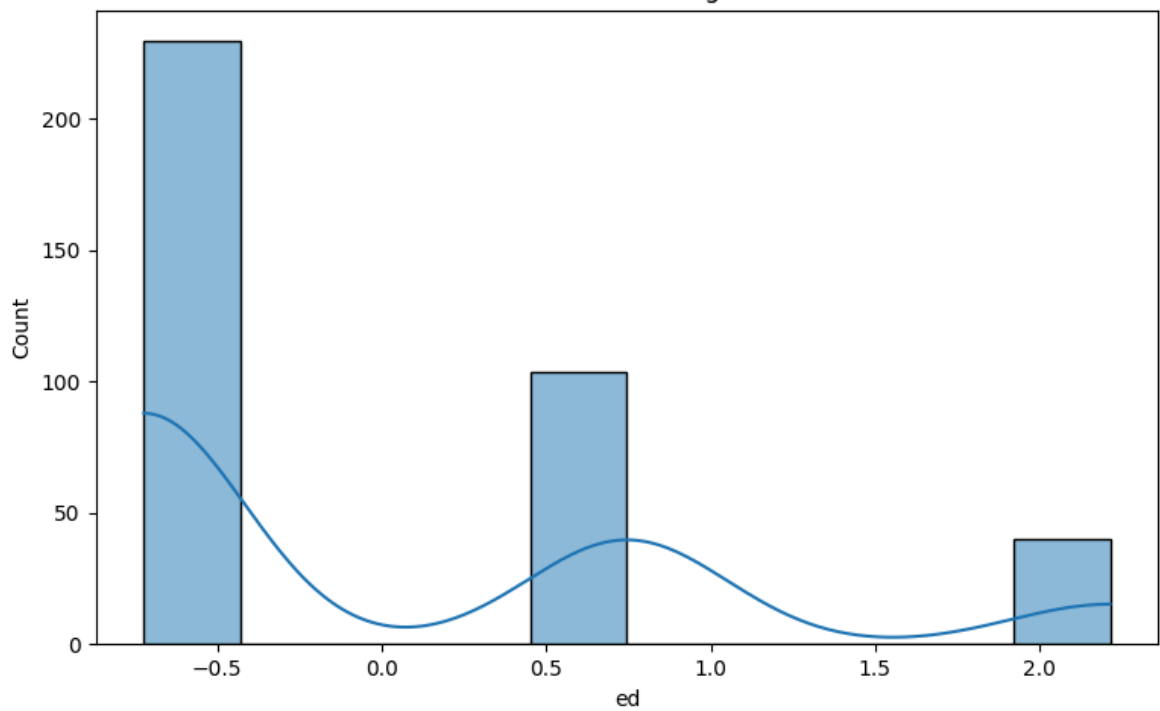




Standard Scaled Histogram - othdebt

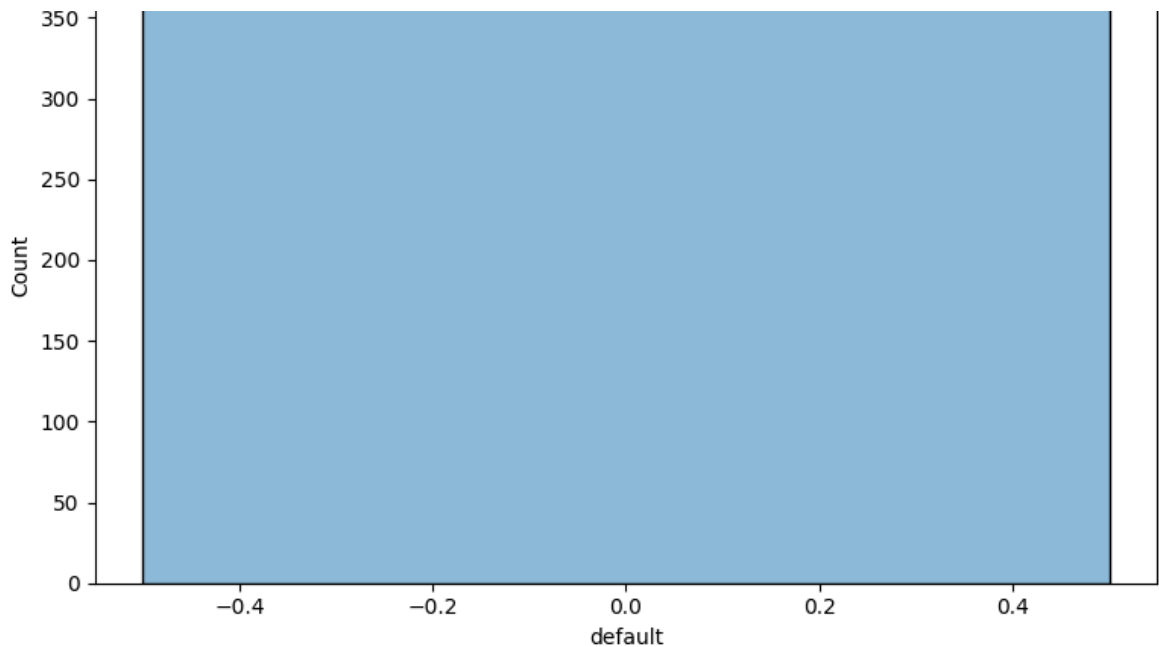


Standard Scaled Histogram - ed



Standard Scaled Histogram - default



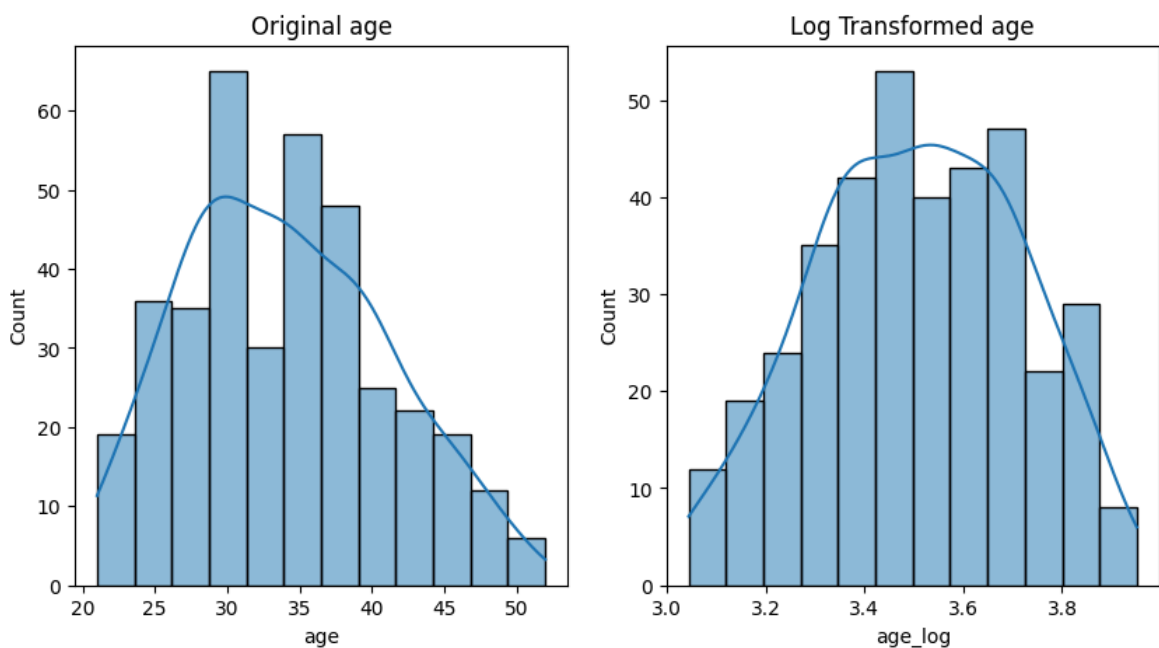


Log transformation

```
In [15]: for col in numeric_columns:
          if (data[col] > 0).all():
              data[col + '_log'] = np.log(data[col])
```

```
In [16]: sample_column = numeric_columns[0]
          if sample_column + '_log' in data.columns:
              plt.figure(figsize=(10, 5))
              plt.subplot(1, 2, 1)
              sns.histplot(data[sample_column], kde=True)
              plt.title(f'Original {sample_column}')

              plt.subplot(1, 2, 2)
              sns.histplot(data[sample_column + '_log'], kde=True)
              plt.title(f'Log Transformed {sample_column}')
              plt.show()
```



Feature Construction


```
In [17]: data_domain_features = data.copy()
```

```
In [18]: data_domain_features['debt_to_income_ratio'] = (  
    data_domain_features['creddebt'] + data_domain_features['othdebt']  
    ) / data_domain_features['income']
```

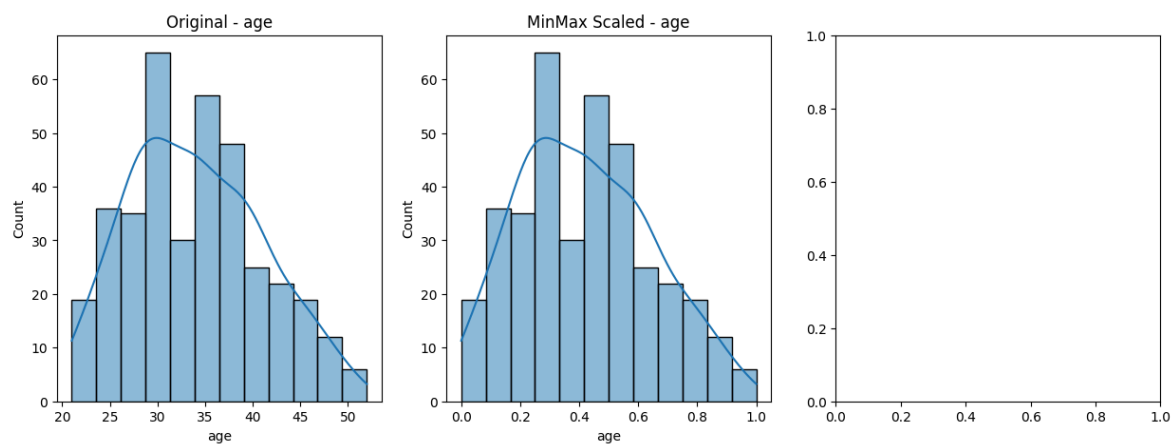
```
In [19]: print("\nDebt-to-Income Ratio (first 20 rows):")  
print(data_domain_features[['creddebt', 'othdebt', 'income', 'debt_to_income_ratio']])
```

```
Debt-to-Income Ratio (first 20 rows):
```

	creddebt	othdebt	income	debt_to_income_ratio
1	1.362202	4.000798	31.0	0.173
5	0.392700	2.157300	25.0	0.102
9	2.777700	2.147300	25.0	0.197
10	0.182512	0.089488	16.0	0.017
11	0.252356	0.943644	23.0	0.052
13	1.715901	3.011099	29.0	0.163
17	1.181952	4.290048	72.0	0.076
18	0.563274	2.913726	61.0	0.057
20	1.154816	0.509184	52.0	0.032
21	0.587552	1.820448	43.0	0.056
22	0.431600	2.168400	26.0	0.100
23	0.402192	2.243808	27.0	0.098
27	0.951232	9.736768	64.0	0.167
28	3.084208	7.587792	58.0	0.184
29	0.204906	5.049094	37.0	0.142
30	0.105000	0.315000	20.0	0.021
31	1.138830	1.171170	22.0	0.105
33	1.018656	2.869344	27.0	0.144
34	0.077140	0.937860	35.0	0.029
36	1.409716	2.132284	22.0	0.161

```
In [20]: sample_col = numeric_columns[0]  
plt.figure(figsize=(15, 5))  
  
plt.subplot(1, 3, 1)  
sns.histplot(data[sample_col], kde=True)  
plt.title(f"Original - {sample_col}")  
  
plt.subplot(1, 3, 2)  
sns.histplot(data_minmax[sample_col], kde=True)  
plt.title(f"MinMax Scaled - {sample_col}")  
  
plt.subplot(1, 3, 3)  
sns.histplot(data_log[sample_col], kde=True)  
plt.title(f"Log Transformed - {sample_col}")  
  
plt.show()
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[20], line 13  
    10 plt.title(f"MinMax Scaled - {sample_col}")  
    12 plt.subplot(1, 3, 3)  
--> 13 sns.histplot(data_log[sample_col], kde=True)  
    14 plt.title(f"Log Transformed - {sample_col}")  
    16 plt.show()  
  
NameError: name 'data_log' is not defined
```



In []: